PRACTICAL-13

PART A:

1. Generic containers: a generic box with get/set and a Pair; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

CODE:

```
package lab13;

class Box<T> {

private T value;


    public void set(T value) {

this.value = value;

    }

    public T get() {

return value;

    }

}

class Pair<A, B> {

    private A first;

private B second;


    public Pair(A first, B second) {

        this.first = first;

        this.second = second;

    }
```

```
    public A getFirst() {
return first;
    }

    public B getSecond() {
return second;
    }
}

public class GenericDemo {

    public static <T> T firstElement(T[] array) {
return array[0];
    }
    public static void main(String[] args) {
Box<Integer> intBox = new Box<>();
intBox.set(100);

        Box<String> stringBox = new Box<>();
stringBox.set("Hello");

        System.out.println("Integer Box: " + intBox.get());
        System.out.println("String Box: " + stringBox.get());

        Pair<String, Integer> student =
```

```
            new Pair<>("Krisha", 21);

        System.out.println("Name: " + student.getFirst());
        System.out.println("Age: " + student.getSecond());

        String[] names = {"Diya", "Krisha", "Ishu"};
        Integer[] numbers = {10, 20, 30};

        System.out.println("First name: " + firstElement(names));
        System.out.println("First number: " + firstElement(numbers));
    }
}
```

OUTPUT:

```
Integer Box: 100
String Box: Hello
Name: Krisha
Age: 21
First name: Diya
First number: 10
PS C:\Users\Krish\OneDrive\Documents\JAVA-practicals>
```

LEARNING OUTCOMES:

- Understand the concept of generics in Java and their advantages.
- Learn to create generic classes such as Box<T> and Pair<A, B>.

- Use generic containers with different data types while maintaining type safety.
- Implement a generic method to return the first element of an array.
- Reduce the need for explicit type casting using generics.

2. Bounded stats: a generic method double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses nonNumber types.

CODE:

```
package lab13; public class BoundedStats {     public static <N
extends Number> double average(N[] values) {        double sum
= 0;        for (N value : values) {            sum +=
value.doubleValue();
      }

      return sum / values.length;
   }

   public static void main(String[] args) {

      Integer[] integers = {10, 20, 30, 40};
      Double[] doubles = {10.5, 20.5, 30.5};
      Long[] longs = {100L, 200L, 300L};
      System.out.println("Integer average: " + average(integers));
      System.out.println("Double average: " + average(doubles));
```

```
        System.out.println("Long average: " + average(longs));
    }
}
```

OUTPUT:

```
Integer average: 25.0
Double average: 20.5
Long average: 200.0
PS C:\Users\Krish\OneDrive\Documents\JAVA-practicals>
```

LEARNING OUTCOMES:

- Understand bounded type parameters using extends in Java generics.
- Learn to create a generic method that accepts numeric types using Number.
- Calculate the average of Integer, Double, and Long arrays.
- Use the doubleValue() method to perform calculations on different numeric types.
- Understand why non-numeric types are rejected by the bounded type parameter.

3. Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.

CODE:

```
package lab13;

class MyStack<T> {
```

```
    private T[] stack;
private int top;

    @SuppressWarnings("unchecked")
    public MyStack(int size) {
stack = (T[]) new Object[size];
top = -1;
    }

    public void push(T value) {

        if (top == stack.length - 1) {
System.out.println("Stack is full!");
return;
        }
        stack[++top] = value;
    }

    public T pop() {

        if (top == -1) {
            System.out.println("Stack is empty!");            return null;
        }
        return stack[top--];
    }
```

```
    public T peek() {

        if (top == -1) {
            System.out.println("Stack is empty!");
return null;
        }
        return stack[top];
    }

    public boolean isEmpty() {
return top == -1;
    }
}
public class GenericStackDemo {

    public static void main(String[] args) {

        MyStack<Integer> intStack = new MyStack<>(5);

        intStack.push(10);
intStack.push(20);
intStack.push(30);

        System.out.println("Integer stack pop: "
                + intStack.pop());
```

```
        System.out.println("Integer stack peek: "
                + intStack.peek());

        MyStack<String> stringStack = new MyStack<>(5);

        stringStack.push("Java");
stringStack.push("Python");
stringStack.push("C++");

        System.out.println("String stack pop: "
                + stringStack.pop());

        System.out.println("String stack peek: "
                + stringStack.peek());
while (!stringStack.isEmpty()) {
stringStack.pop();
        }
        System.out.println("Trying to pop from empty stack:");
stringStack.pop();
    }
}
```

OUTPUT:

```
Integer stack pop: 30
Integer stack peek: 20
String stack pop: C++
String stack peek: Python
Trying to pop from empty stack:
Stack is empty!
PS C:\Users\Krish\OneDrive\Documents\JAVA-practicals>
```

LEARNING OUTCOMES:

- Understand the implementation of a stack using a generic class and an array.
- Learn to perform stack operations such as push(), pop(), and peek().
- Reuse the same generic stack implementation for different data types.
- Handle stack overflow and underflow conditions appropriately.
- Understand how generics provide type safety and code reusability.